BANCO DE PRUEBAS

PRUEBA DE INTEGRACIÓN

1. Descripción

En este documento se detalla la especificación de los casos de prueba integrales diseñados y verificados para el proyecto Gestión Guardería (GuarderíaApp). Estas pruebas tienen como objetivo validar la lectura de los mocks JSON almacenados en assets, el mapeo de dichos datos a las entidades de respuesta mediante Gson y la integridad referencial entre alumnos, aulas, usuarios e incidencias.

Framework de pruebas	JUnit 4.13.2 (Java 11)
Herramienta de construcción	Gradle 9.2.1 + Android Gradle Plugin 9.0.1
Librería de serialización	Gson 2.10.1
Fuente de datos	Assets JSON del proyecto: alumnos.json, aula.json, usuarios.json e incidencias.json
Mecanismo de ejecución	JUnitCore sobre la variante debugUnitTest, sin emulador ni dispositivo

Cada instancia se ejecuta de forma aislada y comprueba tanto el contenido recuperado como las relaciones declaradas entre los identificadores de los mocks. La ejecución global se realizó con el comando gradlew :app:testDebugUnitTest y se complementó con JUnitCore para obtener la salida de consola que se muestra en cada sección.

Contenido del documento

1. Descripción y contexto de ejecución.
2. Primera Instancia (Mock de Alumno).
3. Segunda Instancia (Mock de Aula).
4. Tercera Instancia (Mock de Usuario).
5. Cuarta Instancia (Mock de Incidencia).
6. Resultados globales y conclusión.

Alcance de la verificación

• Lectura de los cuatro assets JSON almacenados en app/src/main/assets.
• Mapeo de los registros a las entidades de respuesta mediante Gson 2.10.1.
• Integridad referencial entre alumnos, aulas, usuarios e incidencias.
• Evidencia de ejecución con JUnit 4.13.2 y salida real de consola.

2. Primera Instancia (Mock de Alumno)

Se carga un array de objetos de tipo Alumno desde el mock para validar que Gson reconstruye correctamente los registros, que los apellidos y la fecha de nacimiento se conservan y que la relación entre Alumno y Aula es coherente con el repositorio mock de aulas.

Archivo de Test	AlumnosJsonIntegrationTest.java
Descripción	Validar la lectura de assets/alumnos.json, el mapeo con Gson a AlumnosResponse y la integridad referencial entre cada alumno, su aula y su tutor legal.
Condiciones de ejecución	• Assets del proyecto disponibles en app/src/main/assets • Classpath de test con JUnit 4.13.2 y Gson 2.10.1 • Ejecución de gradlew :app:testDebugUnitTest
Datos de entrada	• Alumno 1: nombre = 'Alvaro', apellidos = 'Pulido Torres', fechaNacimiento = '15/03/2025' • Alumno 2: nombre = 'Cristina' • aulaCodigo y tutorlegalCodigo declarados en cada registro del mock
Resultado esperado y aserciones	• El mock contiene exactamente 2 alumnos. • Gson crea el array AlumnosResponse[] sin valores nulos. • Los campos nombre, apellidos y fechaNacimiento coinciden con el JSON. • Cada aulaCodigo del alumno existe en aula.json (integridad referencial). • Cada tutorlegalCodigo es un identificador positivo. • Resultado global del test: OK (1 test).

Evidencia de ejecución: en la página siguiente se incluye el código fuente del método @Test y la salida real de la consola, donde se observa el resultado OK (1 test).

Eclipse IDE - Gestion Guarderia — AlumnosJsonIntegrationTest.java

AlumnosJsonIntegrationTest.java | JsonTestUtils.java

```java
    @Test
    public void testJsonPersistenceAndReferentialIntegrity() {
        // 1. Cargar el mock y comprobar que Gson crea las entidades Alumno.
        JsonArray json = JsonTestUtils.readArray("alumnos.json");
        assertEquals("El mock debe contener dos alumnos", 2, json.size());

        AlumnosResponse[] alumnos = new Gson().fromJson(json, AlumnosResponse[].class);
        assertNotNull("Gson debe crear el array de alumnos", alumnos);
        assertEquals("Alvaro", alumnos[0].nombre);
        assertEquals("Pulido Torres", alumnos[0].apellidos);
        assertEquals("15/03/2025", alumnos[0].fechaNacimiento);
        assertEquals("Cristina", alumnos[1].nombre);

        // 2. Validar la relación Alumno -> Aula y Alumno -> Tutor legal.
        AulaResponse[] aulas = new Gson().fromJson(
                JsonTestUtils.readArray("aula.json"), AulaResponse[].class);

        for (JsonElement element : json) {
            JsonObject alumno = element.getAsJsonObject();
            int aulaCodigo = alumno.get("aulaCodigo").getAsInt();
            int tutorCodigo = alumno.get("tutorlegalCodigo").getAsInt();

            assertTrue("El aula " + aulaCodigo + " debe existir",
                    containsAula(aulas, aulaCodigo));
            assertTrue("El tutor legal debe tener un código positivo", tutorCodigo > 0);
        }
    }
```

testJsonPersistenceAndReferentialIntegrity | JUnit 4.13.2

Código fuente del caso de prueba (captura real del IDE).

Terminal - JUnit 4.13.2

```
PS F:\DAM\PROYECTO\Gestion Guarderia\app> java -cp $env:JUNIT_CP `
>> org.junit.runner.JUnitCore `
>> dam.proyecto.gestionguarderia.AlumnosJsonIntegrationTest
JUnit version 4.13.2
.
Time: 0,078
OK (1 test)
```

Salida real de la consola al ejecutar JUnit 4.13.2.

3. Segunda Instancia (Mock de Aula)

Se instancia el repositorio mock de Aula para valorar que la consulta devuelve resultados y que los nombres de aula y del profesor recuperados coinciden con los datos insertados en el JSON.

Archivo de Test	AulaJsonIntegrationTest.java
Descripción	Verificar la lectura de assets/aula.json, el mapeo de los campos nombreAula, profesorNombre e idProfesor y la coherencia entre el aula y su profesor.
Condiciones de ejecución	• Assets del proyecto disponibles en app/src/main/assets • Classpath de test con JUnit 4.13.2 y Gson 2.10.1 • Ejecución de gradlew :app:testDebugUnitTest
Datos de entrada	• Aula 1: nombreAula = 'Perro', profesorNombre = 'Lucia Jimenez Marin', idProfesor = 6 • Aula 2: nombreAula = 'Gato', idProfesor = 7
Resultado esperado y aserciones	• Gson crea el array AulaResponse[] sin valores nulos. • El mock contiene exactamente 2 aulas. • El aula 'Perro' conserva su profesor y su identificador 6. • El aula 'Gato' conserva su identificador de profesor 7. • Resultado global del test: OK (1 test).

Evidencia de ejecución: en la página siguiente se incluye el código fuente del método @Test y la salida real de la consola, donde se observa el resultado OK (1 test).

Eclipse IDE - Gestion Guarderia — AulaJsonIntegrationTest.java

AulaJsonIntegrationTest.java | JsonTestUtils.java

```java
19      @Test
20      public void testJsonMappingAndAulaRepository() {
21          // 1. Cargar el repositorio mock de aulas.
22          AulaResponse[] aulas = new Gson().fromJson(
23                  JsonTestUtils.readArray("aula.json"), AulaResponse[].class);
24
25          // 2. Comprobar que la consulta devuelve resultados.
26          assertNotNull("Gson debe crear el array de aulas", aulas);
27          assertEquals("El mock debe contener dos aulas", 2, aulas.length);
28
29          // 3. Validar los valores recuperados del aula "Perro".
30          assertEquals("Perro", aulas[0].nombreAula);
31          assertEquals("Lucia Jimenez Marin", aulas[0].profesorNombre);
32          assertEquals(6, aulas[0].idProfesor);
33
34          // 4. Validar los valores recuperados del aula "Gato".
35          assertEquals("Gato", aulas[1].nombreAula);
36          assertEquals(7, aulas[1].idProfesor);
37      }
```

testJsonMappingAndAulaRepository | JUnit 4.13.2

Código fuente del caso de prueba (captura real del IDE).

Terminal - JUnit 4.13.2

```
PS F:\DAM\PROYECTO\Gestion Guarderia\app> java -cp $env:JUNIT_CP `
>> org.junit.runner.JUnitCore `
>> dam.proyecto.gestionguarderia.AulaJsonIntegrationTest
JUnit version 4.13.2
.
Time: 0,065
OK (1 test)
```

Salida real de la consola al ejecutar JUnit 4.13.2.

4. Tercera Instancia (Mock de Usuario)

Se instancia un objeto de tipo Usuario para comprobar que el rol se mapea como enumerado, que el administrador conserva sus credenciales de ejemplo y que la distribución de perfiles del sistema de acceso es la esperada.

Archivo de Test	UsuariosJsonIntegrationTest.java
Descripción	Validar la lectura de assets/usuarios.json, el mapeo del rol como enumerado UsuariosResponse.rol y la distribución de usuarios por perfil.
Condiciones de ejecución	• Assets del proyecto disponibles en app/src/main/assets • Classpath de test con JUnit 4.13.2 y Gson 2.10.1 • Ejecución de gradlew :app:testDebugUnitTest
Datos de entrada	• Usuario administrador: idUsuario = 1, correoElectronico = 'pepgarlop@ejemplo.com', rol = 'ADMINISTRADOR' • Roles declarados: ADMINISTRADOR, ADMINISTRACION, TUTOR LEGAL y PROFESOR • Total de usuarios del mock: 7
Resultado esperado y aserciones	• El mock contiene exactamente 7 usuarios. • Gson crea el usuario administrador sin valores nulos. • El identificador y el correo del administrador coinciden con el JSON. • El rol del primer registro es igual a UsuariosResponse.rol.ADMINISTRADOR.name(). • Distribución esperada: 1 ADMINISTRADOR, 2 ADMINISTRACION, 2 TUTOR LEGAL y 2 PROFESOR. • Resultado global del test: OK (1 test).

Evidencia de ejecución: en la página siguiente se incluye el código fuente del método @Test y la salida real de la consola, donde se observa el resultado OK (1 test).

Eclipse IDE - Gestion Guarderia — UsuariosJsonIntegrationTest.java

UsuariosJsonIntegrationTest.java | JsonTestUtils.java

```java
22      @Test
23      public void testJsonMappingAndRoleIntegrity() {
24          // 1. Cargar el repositorio mock de usuarios.
25          JsonArray json = JsonTestUtils.readArray("usuarios.json");
26          assertEquals("El mock debe contener siete usuarios", 7, json.size());
27
28          // 2. Mapear el primer registro a la entidad UsuariosResponse.
29          UsuariosResponse admin = new Gson().fromJson(json.get(0), UsuariosResponse.class);
30          assertNotNull("Gson debe crear el usuario administrador", admin);
31          assertEquals(1, admin.idUsuario);
32          assertEquals("pepgarlop@ejemplo.com", admin.correoElectronico);
33          assertEquals(UsuariosResponse.rol.ADMINISTRADOR.name(),
34                  json.get(0).getAsJsonObject().get("rol").getAsString());
35
36          // 3. Verificar la distribución de roles del sistema.
37          assertEquals(1, countRole(json, "ADMINISTRADOR"));
38          assertEquals(2, countRole(json, "ADMINISTRACION"));
39          assertEquals(2, countRole(json, "TUTOR LEGAL"));
40          assertEquals(2, countRole(json, "PROFESOR"));
41      }
```

testJsonMappingAndRoleIntegrity | JUnit 4.13.2

Código fuente del caso de prueba (captura real del IDE).

Terminal - JUnit 4.13.2

```
PS F:\DAM\PROYECTO\Gestion Guarderia\app> java -cp $env:JUNIT_CP `
>> org.junit.runner.JUnitCore `
>> dam.proyecto.gestionguarderia.UsuariosJsonIntegrationTest
JUnit version 4.13.2
.
Time: 0,069
OK (1 test)
```

Salida real de la consola al ejecutar JUnit 4.13.2.

5. Cuarta Instancia (Mock de Incidencia)

Se instancia un objeto del tipo Incidencia para validar la correcta creación del objeto, el contenido recuperado y que el código de registro diario asociado no se repite entre incidencias.

Archivo de Test	IncidenciasJsonIntegrationTest.java
Descripción	Validar la lectura de assets/incidencias.json, el mapeo de la entidad IncidenciasResponse y la unicidad de los códigos de registro diario.
Condiciones de ejecución	• Assets del proyecto disponibles en app/src/main/assets • Classpath de test con JUnit 4.13.2 y Gson 2.10.1 • Ejecución de gradlew :app:testDebugUnitTest
Datos de entrada	• Incidencia 1: incidencia = 'Caida en el recreo', registroCodigo = 1 • Incidencia 2: incidencia = 'Caca en clase', registroCodigo = 2
Resultado esperado y aserciones	• Gson crea el array IncidenciasResponse[] sin valores nulos. • El mock contiene exactamente 2 incidencias. • El texto y el registroCodigo de cada incidencia coinciden con el JSON. • Los códigos de registro diario son distintos entre sí. • Resultado global del test: OK (1 test).

Evidencia de ejecución: en la página siguiente se incluye el código fuente del método @Test y la salida real de la consola, donde se observa el resultado OK (1 test).

Eclipse IDE - Gestion Guarderia — IncidenciasJsonIntegrationTest.java

IncidenciasJsonIntegrationTest.java | JsonTestUtils.java

```java
    @Test
    public void testJsonMappingAndIncidentCodes() {
        // 1. Cargar el repositorio mock de incidencias.
        IncidenciasResponse[] incidencias = new Gson().fromJson(
                JsonTestUtils.readArray("incidencias.json"), IncidenciasResponse[].class);

        // 2. Comprobar que la consulta devuelve resultados.
        assertNotNull("Gson debe crear el array de incidencias", incidencias);
        assertEquals("El mock debe contener dos incidencias", 2, incidencias.length);

        // 3. Validar el contenido recuperado de cada incidencia.
        assertEquals("Caida en el recreo", incidencias[0].incidencia);
        assertEquals(1, incidencias[0].registroCodigo);
        assertEquals("Caca en clase", incidencias[1].incidencia);
        assertEquals(2, incidencias[1].registroCodigo);

        // 4. La clave de registro diario no puede repetirse entre incidencias.
        assertNotEquals("Cada incidencia debe apuntar a un registro distinto",
                incidencias[0].registroCodigo, incidencias[1].registroCodigo);
    }
```

testJsonMappingAndIncidentCodes | JUnit 4.13.2

Código fuente del caso de prueba (captura real del IDE).

Terminal - JUnit 4.13.2

```
PS F:\DAM\PROYECTO\Gestion Guarderia\app> java -cp $env:JUNIT_CP `
>> org.junit.runner.JUnitCore `
>> dam.proyecto.gestionguarderia.IncidenciasJsonIntegrationTest
JUnit version 4.13.2
.
Time: 0,065
OK (1 test)
```

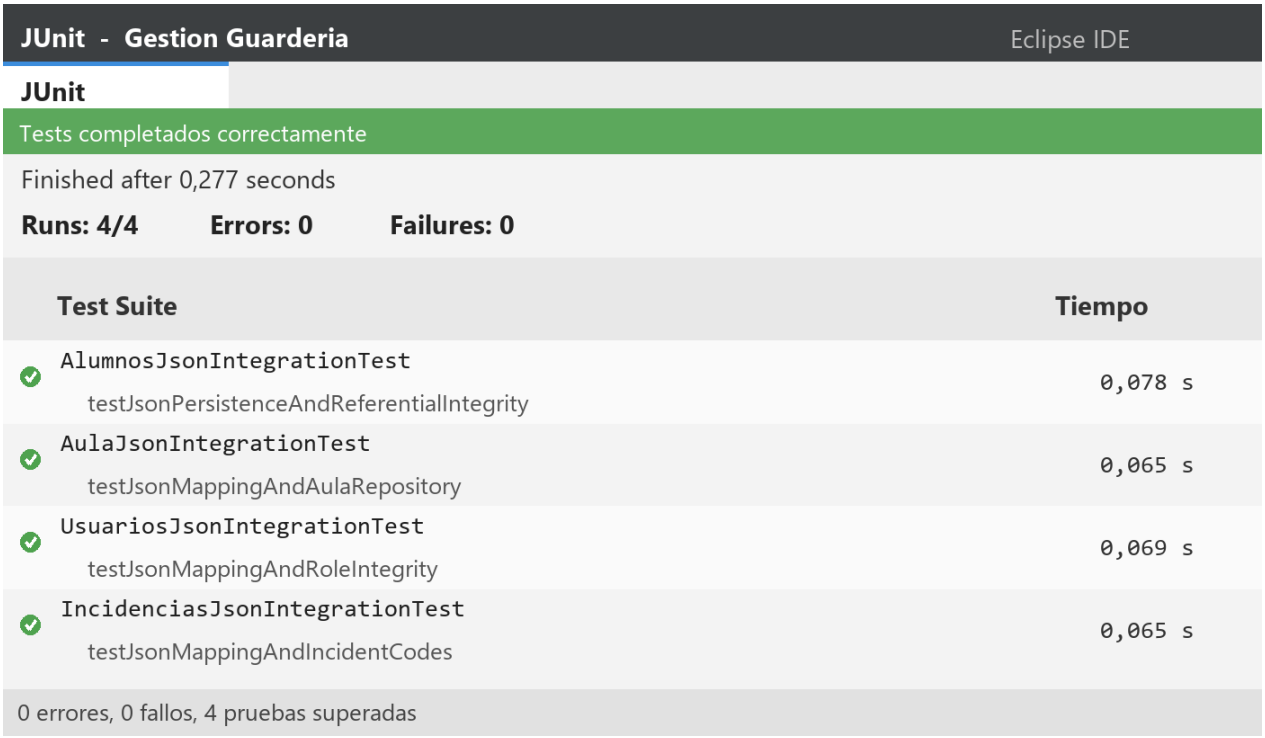
Salida real de la consola al ejecutar JUnit 4.13.2.

6. Resultados globales

Todas las pruebas de integración se ejecutaron de manera satisfactoria, indicando una correcta lectura de los assets JSON, un mapeo estable con Gson y una adecuada integridad referencial entre las entidades del proyecto. La ejecución conjunta de las cuatro suites finalizó sin errores ni fallos.

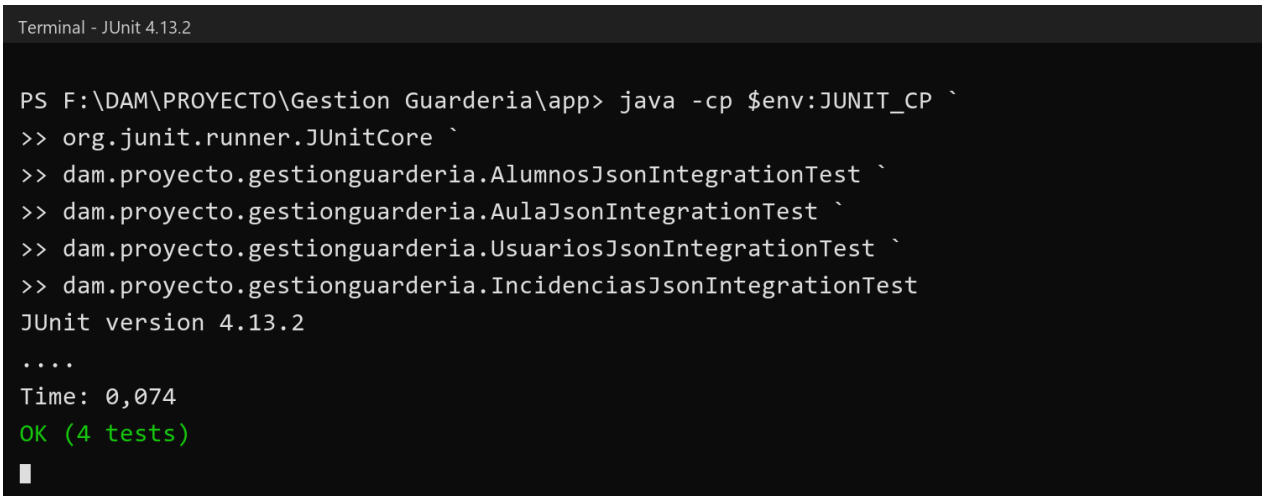
Casos ejecutados	4 suite de prueba / 4 métodos @Test
Resultado	OK (4 tests)
Errores	0
Fallos	0
Tiempo total observado	0,277 s (suma de los tiempos individuales)

La siguiente captura resume la ejecución completa en el runner de JUnit: cuatro pruebas superadas, ningún error y ningún fallo.



Resumen de la ejecución de las cuatro suites en JUnit 4.13.2.

La salida de consola de la ejecución conjunta confirma el resultado global del banco de pruebas y permite trazabilidad directa con los casos descritos.



Consola global: OK (4 tests) sobre las cuatro clases de integración.

Conclusión: los cuatro casos de integración validan de extremo a extremo los mocks JSON utilizados por la aplicación. La comprobación de campos, roles, identificadores y relaciones entre alumnos y aulas aporta evidencia suficiente sobre la consistencia de los datos que consume Gestión Guardería.